To add the persistence API to your project, add the following to build.sbt. You will be testing the application, so also add the specific test kit:

```
val AkkaVersion = "2.6.20"
libraryDependencies ++= Seq(
  "com.typesafe.akka" %% "akka-persistence-typed" % AkkaVersion,
  "com.typesafe.akka" %% "akka-persistence-testkit" % AkkaVersion % Test
)
```

NOTE The source code for this example is available at www.manning.com/books/akka-in-action-second-edition or https://github.com/franciscolopez sancho/akka-topics/tree/main/chapter09b. You can find the contents of any snippet or listing in the .scala file with the same name as the class, object, or trait.

To create a persistent actor, you need to create a specific behavior of type `EventSourcedBehavior`. Its constructor takes four parameters (`persistenceId`, `state`, `commandHandler`, and `eventHandler`) and three types (`Command`, `Event`, and `State`). All of them together make an `EventSourcedBehavior`:

```
import akka.persistence.typed.scaladsl.EventSourcedBehavior
import akka.persistence.typed.PersistenceId

object PersistentActorExample {

  def apply[Command, Event, State](                          ❶

    persistenceId: PersistenceId,                            ❷

    state: State,                                            ❸

    commandHandler: (State, Command) => Effect[Event, State],  ❹

    eventHandler: (State, Event) => State)
           :EventSourcedBehavior[Command, Event, State]        ❺

}
```

❶ Generic types with descriptive meanings
❷ ID used to keep a reference to the actor when the actor is stored
❸ Initial state of the persistent actor
❹ Function that applies a command to a state, producing an event and a state
❺ Function that stores an event with a state

The names of the types are just that: names. They are not types that exist outside of this definition, with the exception of the `State` type, which refers to a class within the actor. Nevertheless, it is common to use the same names when creating a behavior. Let's look at the four parameters.

The `persistenceId` must be unique. Note that there are scenarios where collisions can occur. For example, in a cluster, you can create an actor with the same ID in different nodes. Akka Persistence doesn't have a mechanism to check the uniqueness of a single node, but Akka